Autoencoders (AEs) and Variational Autoencoders (VAEs)

Unsupervised Representation Learning Models

Previous session

Limitations of autoencoders

Variational Autoencoder:

- Architecture
- Loss function

Today's Session Overview

- Reparameterization Trick

Variational Autoencoders (VAEs): Loss Function

A VAE does two things:

1. Reconstruct the input well (like a normal autoencoder)
2. Force the latent space to follow a known distribution (so we can *generate* new data)

That is why the loss has two terms:

$$\mathcal{L}(x) = -\mathbb{E}_{q(z \mid x)}[\log p(x \mid z)] + KL\big(q(z \mid x) \parallel p(z)\big)$$

Reconstruction / Data consistency *Regularization*

Mean square error

$$L = L_{KL}(\mathcal{N}(\mu, \sigma^2) \parallel \mathcal{N}(0,1))$$

$$L_{KL} = \frac{1}{2}[\mu^2 + \sigma^2 - log\sigma^2 - 1)$$

For the 2 gaussian distribution, the KL divergence will take this form

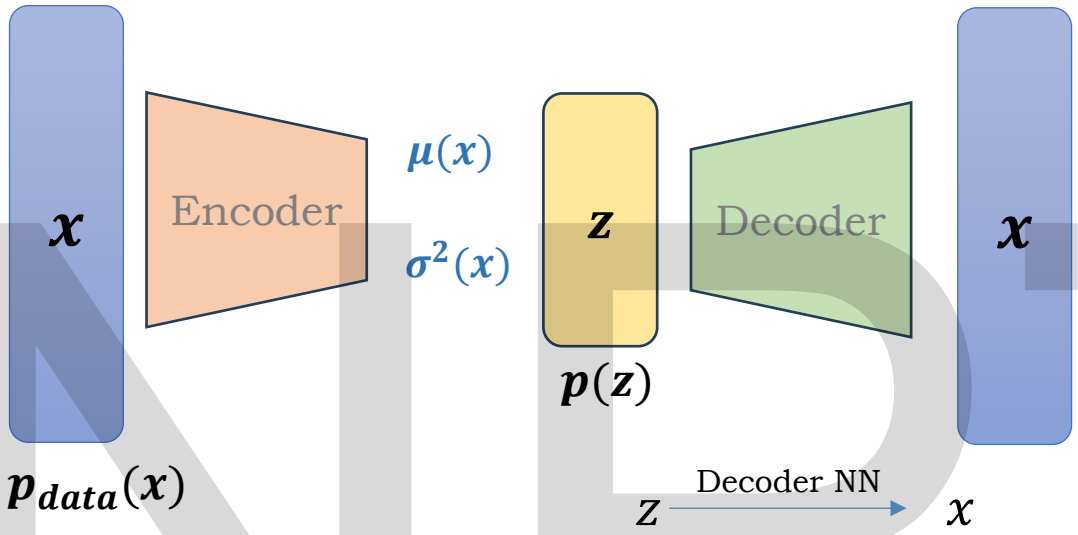
Loss function depends on a random variable z.
z is sampled from a distribution:

$$z \sim q(z \mid x) \sim \mathcal{N}(\mu(x), \sigma^2(x))$$

- Sampling is a random operation
- Random sampling breaks gradient flow
- Neural networks are trained using gradient descent
- Backpropagation cannot pass through randomness

So, the question is: **How do we compute gradients when part of the network involves sampling?**

This motivates the Reparameterization Trick



Once z is sampled:

$$x = f_\theta(z)$$

Then:

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial x} \cdot \frac{\partial x}{\partial w}$$

$$\frac{\partial L}{\partial b} = \frac{\partial L}{\partial x} \cdot \frac{\partial x}{\partial b}$$

Once z is sampled, the decoder behaves like a normal neural network.

Therefore, gradients w.r.t. decoder parameters are computed using standard backpropagation and are never the problem in VAEs.

All learnable quantities inside this network are decoder parameters.

Fully connected (MLP) decoder

Suppose the decoder is:

$$h_1 = ReLU(W_1 z + b_1)$$

$$x = \sigma(W_2 h_1 + b_2)$$

Decoder parameters:

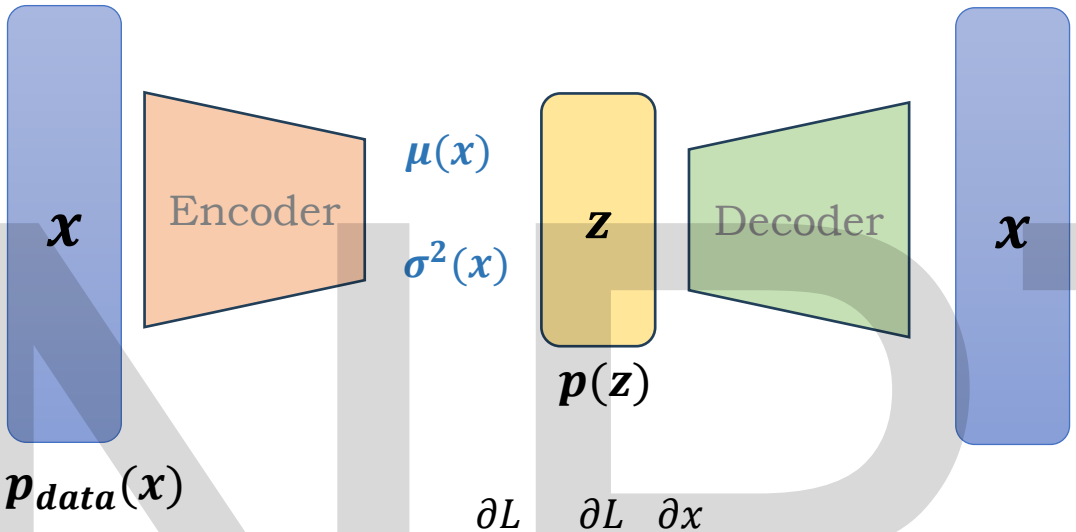
$$\theta = \{W_1, b_1, W_2, b_2\}$$

CNN-based decoder (deconvolution / upsampling)

If the decoder uses ConvTranspose or Upsampling + Conv:

- Convolution kernels
- Bias terms of conv layers
- BatchNorm parameters (if used)

All of these together are **decoder parameters** θ.



$$\frac{\partial L}{\partial z} = \frac{\partial L}{\partial x} \cdot \frac{\partial x}{\partial z}$$

This is also fine because:

- z is just a numeric vector at this point
- The decoder is differentiable w.r.t. its input

So:

- Gradients **flow up to** z without any difficulty

The problem starts after gradients reach z.

z was obtained by random sampling from a distribution parameterized by μ and σ

$$z \sim q(z \mid x) \sim \mathcal{N}(\mu(x), \sigma(x)^2)$$

To compute:

$$\frac{\partial L}{\partial \mu} \text{ and } \frac{\partial L}{\partial \sigma}$$

Using chain rule:

$$\frac{\partial L}{\partial \mu} = \frac{\partial L}{\partial z} \cdot \frac{\partial z}{\partial \mu}$$

$$\frac{\partial L}{\partial \sigma} = \frac{\partial L}{\partial z} \cdot \frac{\partial z}{\partial \sigma}$$

But here is the issue:
A sampling operation is not a deterministic function of μ, σ

So $\frac{\partial z}{\partial \mu}$ and $\frac{\partial z}{\partial \sigma}$ are not well-defined, because there is no fixed rule that tells how z changes when μ or σ changes, because z is obtained by a random draw, not by a formula).

μ did not change
σ did not change
But still z *changed.*
$\therefore \frac{\partial z}{\partial \mu}$ *is undefined*

What is $\frac{\partial z}{\partial \mu}$

A sampling operation is not a deterministic function of μ and σ

Trial	μ	σ	sampled z
1	1	0.5	0.9
2	1	0.5	1.6
3	1	0.5	0.4

$$z \sim \mathcal{N}(\mu, \sigma^2)$$

Now imagine:

$$\mu = 1 \text{ and } \sigma = 0.5$$

$$z \sim \mathcal{N}(1, 0.25)$$

Now we sample multiple times

How can we sample z in such a way that gradients can flow back to μ and σ?

Instead of directly sampling:

$$z \sim \mathcal{N}(\mu, \sigma^2)$$

Can we generate z using a formula where randomness is independent of μ, σ?

If we can do this:

- ✓ Randomness will no longer block gradients
- ✓ Backpropagation will work end-to-end

The Reparameterization Trick

A technique that rewrites the sampling operation as a deterministic function of μ, σ and an independent random variable.

$$\boldsymbol{z = \mu + \sigma \cdot \varepsilon}, \qquad \boldsymbol{\varepsilon \sim \mathcal{N}(0,1)}$$

where

- μ= mean (from encoder)
- σ= standard deviation (from encoder)
- $\varepsilon \sim \mathcal{N}(0,1)$ (random noise, independent of μ, σ)

During backpropagation, ε is treated as a constant (doesnot depend of network parameters).

Mathematically, while differentiating

$$\boldsymbol{\varepsilon = constant}$$

Compute the partial derivative w.r.t. μ

$$z = \mu + \sigma\varepsilon$$

Differentiate w.r.t. μ:

$$\frac{\partial z}{\partial \mu} = \frac{\partial}{\partial \mu}(\mu) + \frac{\partial}{\partial \mu}(\sigma\varepsilon)$$

- $\frac{\partial \mu}{\partial \mu} = 1$
- $\sigma\varepsilon$ does **not** contain $\mu \rightarrow$ derivative is 0

$$\frac{\partial z}{\partial \mu} = 1$$ ✓

Compute the partial derivative w.r.t. σ

$$z = \mu + \sigma\varepsilon$$

Differentiate w.r.t. σ:

$$\frac{\partial z}{\partial \sigma} = \frac{\partial}{\partial \sigma}(\mu) + \frac{\partial}{\partial \sigma}(\sigma\varepsilon)$$

- μ does not depend on $\sigma \rightarrow$ derivative is 0
- ε is a constant

$$\frac{\partial}{\partial \sigma}(\sigma\varepsilon) = \varepsilon$$

$$\frac{\partial z}{\partial \sigma} = \varepsilon$$ ✓

Chain rule with the loss:

Using chain rule:

$$\frac{\partial L}{\partial \mu} = \frac{\partial L}{\partial z} \cdot \frac{\partial z}{\partial \mu} = \frac{\partial L}{\partial z} \cdot 1$$

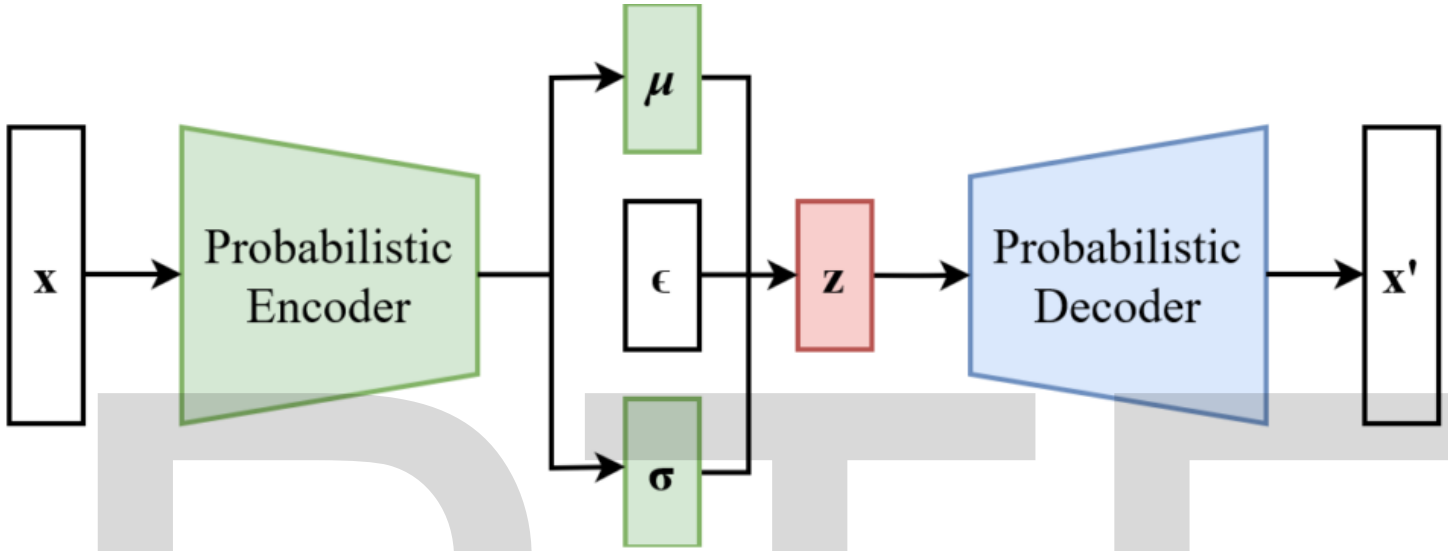
$$\frac{\partial L}{\partial \sigma} = \frac{\partial L}{\partial z} \cdot \frac{\partial z}{\partial \sigma} = \frac{\partial L}{\partial z} \cdot \varepsilon$$

$\frac{\partial L}{\partial z}$ comes from decoder backpropagation

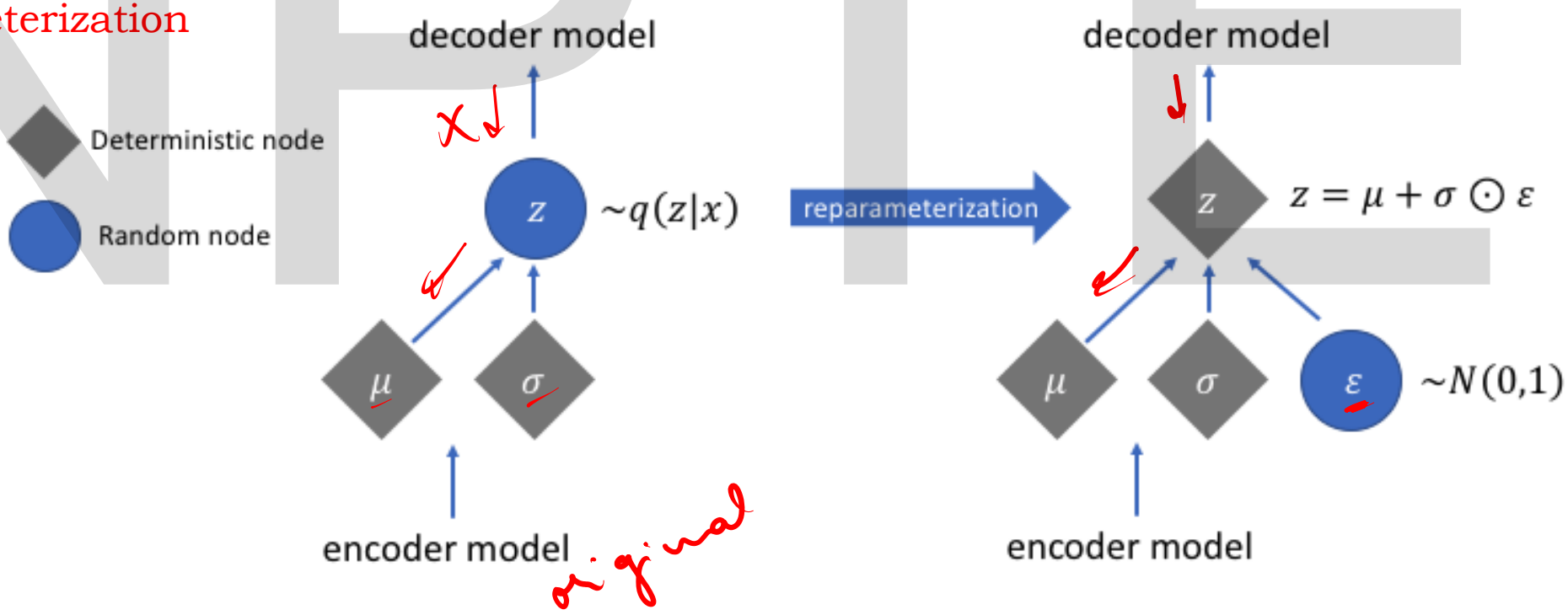
$$\frac{\partial L}{\partial z} = \frac{\partial L}{\partial x} \cdot \frac{\partial x}{\partial z}$$

∴ There is a normal flow of gradient.

VAE Architecture



Reparameterization



Next Session

- Hands-on implementation on Autoencoders

Thank you